

Overview of Autoencoder

Subject: Autoencoder

1.

Definition An autoencoder is defined by the following components: Two sets: the space of encoded messages Z ; the space of decoded messages X . Typically X and Z are Euclidean spaces, that is, $X = \mathbb{R}^m$, $Z = \mathbb{R}^n$ with $m > n$. Two parametrized families of functions: the encoder family $E_\phi : X \rightarrow Z$, parametrized by ϕ ; the decoder family $D_\theta : Z \rightarrow X$, parametrized by θ . For any $x \in X$, we usually write $z = E_\phi(x)$, and refer to it as the code, the latent variable, latent representation, latent vector, etc. Conversely, for any $z \in Z$, we usually write $x' = D_\theta(z)$, and refer to it as the (decoded) message. Usually, both the encoder and the decoder are defined as multilayer perceptrons (MLPs). For example, a one-layer-MLP encoder E_ϕ is:

2.

where δ_{x_i} is the Dirac measure, the quality function is just L^2 loss: $d(x, x') = \|x - x'\|_2^2$, and $\|\cdot\|_2$ is the Euclidean norm. Then the problem of searching for the optimal autoencoder is just a least-squares optimization: $\min_{\theta, \phi} L(\theta, \phi)$, where $L(\theta, \phi) = \frac{1}{N} \sum_{i=1}^N \|x_i - D_\theta(E_\phi(x_i))\|_2^2$ (where $L(\theta, \phi) = \frac{1}{N} \sum_{i=1}^N \|x_i - D_\theta(E_\phi(x_i))\|_2^2$)

3.

or the L1 loss, as $s(\rho, \hat{\rho}) = \|\rho - \hat{\rho}\|_1$, or the L2 loss, as $s(\rho, \hat{\rho}) = \|\rho - \hat{\rho}\|_2^2$. Alternatively, the sparsity regularization loss may be defined without reference to any "desired sparsity", but simply force as much sparsity as possible. In this case, one can define the sparsity regularization loss as $L_{\text{sparse}}(\theta, \phi) = \mathbb{E}_{x \sim \mu_X} [\sum_{k=1}^K w_k \|h_k\|_1]$ where h_k is the activation vector in the k -th layer of the autoencoder. The norm $\|\cdot\|$ is usually the L1 norm (giving the L1 sparse autoencoder) or the L2 norm (giving the L2 sparse autoencoder).

4.

Further reading Bank, Dor; Koenigstein, Noam; Giryes, Raja (2023). "Autoencoders". Machine Learning for Data Science Handbook. Cham: Springer International Publishing. doi:10.1007/978-3-031-24628-9_16. ISBN 978-3-031-24627-2. Goodfellow, Ian; Bengio, Yoshua;

Courville, Aaron (2016). "14. Autoencoders". Deep learning. Adaptive computation and machine learning. Cambridge, Mass: The MIT press. ISBN 978-0-262-03561-3.

5.

Machine translation Autoencoders have been applied to machine translation, which is usually referred to as neural machine translation (NMT). Unlike traditional autoencoders, the output does not match the input - it is in another language. In NMT, texts are treated as sequences to be encoded into the learning procedure, while on the decoder side sequences in the target language(s) are generated. Language-specific autoencoders incorporate further linguistic features into the learning procedure, such as Chinese decomposition features. Machine translation is rarely still done with autoencoders, due to the availability of more effective transformer networks.

6.

There exist representations to the messages that are relatively stable and robust to the type of noise we are likely to encounter; The said representations capture structures in the input distribution that are useful for our purposes. Example noise processes include:

7.

Training Geoffrey Hinton developed the deep belief network technique for training many-layered deep autoencoders. His method involves treating each neighboring set of two layers as a restricted Boltzmann machine so that pretraining approximates a good solution, then using backpropagation to fine-tune the results. Researchers have debated whether joint training (i.e. training the whole architecture together with a single global reconstruction objective to optimize) would be better for deep auto-encoders. A 2015 study showed that joint training learns better data models along with more representative features for classification as compared to the layerwise method. However, their experiments showed that the success of joint training depends heavily on the regularization strategies adopted.

8.

Depth can exponentially reduce the computational cost of representing some functions. Depth can exponentially decrease the amount of training data needed to learn some functions. Experimentally, deep autoencoders yield better compression compared to shallow or linear autoencoders. Depth allows for advantages over traditional methods as one can show that after training single layer linear autoencoders have a latent space whose vectors span the same subspace as the eigenvectors found in Principal component analysis.

9.

Dimensionality reduction Dimensionality reduction was one of the first deep learning applications. For Hinton's 2006 study, he pretrained a multi-layer autoencoder with a stack of RBMs and then used their weights to initialize a deep autoencoder with gradually smaller hidden layers until hitting a bottleneck of 30 neurons. The resulting 30 dimensions of the code yielded a smaller reconstruction error compared to the first 30 components of a principal component analysis (PCA), and learned a representation that was qualitatively easier to interpret, clearly separating data clusters. Reducing dimensions can improve performance on tasks such as classification. Indeed, the hallmark of dimensionality reduction is to

place semantically related examples near each other.

10.

An autoencoder is a type of artificial neural network used to learn efficient codings of unlabeled data (unsupervised learning). An autoencoder learns two functions: an encoding function that transforms the input data, and a decoding function that recreates the input data from the encoded representation. The autoencoder learns an efficient representation (encoding) for a set of data, typically for dimensionality reduction, to generate lower-dimensional embeddings for subsequent use by other machine learning algorithms. Variants exist which aim to make the learned representations assume useful properties. Examples are regularized autoencoders (sparse, denoising and contractive autoencoders), which are effective in learning representations for subsequent classification tasks, and variational autoencoders, which can be used as generative models. Autoencoders are applied to many problems, including facial recognition, feature detection, anomaly detection, and learning the meaning of words. In terms of data synthesis, autoencoders can also be used to randomly generate new data that is similar to the input (training) data.